

Лабораторная работа №2: Компиляция и сравнение быстродействия C++ и Python

Лана Глазко

Май 2025, 3 курс

Цель

Познакомиться с принципами компиляции исходного кода, реализовать вычисления с использованием циклов, условий и функций, сравнить быстродействие C++ и Python при выполнении одинаковых вычислений, а также закрепить навыки работы с системой контроля версий Git и файловой системой Unix.

Задачи

1. Реализовать программу на C++, выполняющую заданное арифметическое выражение в цикле.
2. Аналогично реализовать программу на Python 3.
3. Осуществить замер времени выполнения в обоих случаях.
4. Организовать структуру проекта, включая директории `src`, `build`, `doc`.
5. Настроить игнорирование каталога сборки через `.gitignore`.
6. Зафиксировать изменения в Git через ветку `dev` и подготовить отчёт в `doc`.

Реализация на C++

Создаём структуру проекта:

```
1 mkdir -p lab_02/src lab_02/build lab_02/doc
2 cd lab_02
3
4 touch .gitignore
5 nano .gitignore
```

Содержимое файла `.gitignore`:

```
1 build/
```

Реализация программы на C++ — файл `src/main.cpp`:

```

1 #include <iostream>
2 #include <chrono>
3 #include <cmath>
4
5 using namespace std;
6 using namespace std::chrono;
7
8 double calculate_expression(double x) {
9     return pow(x, 2) - pow(x, 2) + x * 4 - x * 5 + x + x;
10 }
11
12 int main() {
13     while (true) {
14         cout << "Введите количество итераций (или 'q' для выхода): ";
15         string input_n;
16         cin >> input_n;
17
18         if (input_n == "q") break;
19
20         cout << "Введите значение x (или 'q' для выхода): ";
21         string input_x;
22         cin >> input_x;
23
24         if (input_x == "q") break;
25
26         try {
27             int n = stoi(input_n);
28             double x = stod(input_x);
29
30             auto start = high_resolution_clock::now();
31
32             for (int i = 0; i < n; ++i) {
33                 calculate_expression(x);
34             }
35
36             auto end = high_resolution_clock::now();
37             duration<double> duration = end - start;
38
39             cout << "Время выполнения: " << duration.count() << " секунд\n\n";
40
41         } catch (invalid_argument&) {
42             cout << "Ошибка: введено не число. Завершение программы.\n";
43             break;
44         }
45     }
46     return 0;
47 }

```

Компиляция программы:

```
1 g++ src/main.cpp -o build/cpp_expr
```

Проверим содержимое папки build:

```
1 ls build
```

Результат:

```
1 cpp_expr
```

Файл `cpp_expr` — это скомпилированный исполняемый файл на C++, который можно запускать из терминала. Его поведение соответствует логике, заложенной в `main.cpp`.

Пример выполнения программы:

```
1 $ ./build/cpp_expr
2 {Введите количество итераций (или 'q' для выхода): 2001
3 Введите значение x (или 'q' для выхода): 2222
4 Время выполнения: 0.000229316 секунд
5
6 Введите количество итераций (или 'q' для выхода): 2001
7 Введите значение x (или 'q' для выхода): g
8 Ошибка: введено не число. Завершение программы.}
```

Реализация на Python

Для реализации аналогичной функциональности на языке Python 3 создаём файл `src/main.py` со следующим содержанием:

```
1 import time
2
3 def calculate_expression(x):
4     return x**2 - x**2 + x*4 - x*5 + x + x
5
6 def main():
7     while True:
8         iterations = input("Введите количество итераций (или 'q' для выхода): ")
9         if iterations == 'q':
10             break
11
12         x_input = input("Введите значение x (или 'q' для выхода): ")
13         if x_input == 'q':
14             break
15
16         try:
17             n = int(iterations)
18             x = float(x_input)
19
20             start_time = time.time()
21
22             for _ in range(n):
23                 calculate_expression(x)
24
25             end_time = time.time()
26             print(f"Время выполнения: {end_time - start_time} секунд\n")
27
28         except ValueError:
29             print("Ошибка: введено не число. Завершение программы.")
```

```

30         break
31
32 if __name__ == "__main__":
33     main()

```

Данный код реализует следующую логику:

- Запрашивает у пользователя количество итераций и значение переменной x ;
- Выполняет арифметические вычисления в цикле указанное количество раз;
- Замеряет и выводит общее время выполнения;
- Обрабатывает ввод некорректных значений (например, символов вместо чисел).

Запуск и результаты

Переходим в директорию проекта и запускаем программу:

```

1 sudbreak@LAPTOP-R4CB5439:/mnt/c/Users/lana/OneDrive/Документы/cpp_unix_project/
  lab_02$ python3 src/main.py

```

Пример корректного выполнения:

```

1 Введите количество итераций (или 'q' для выхода): 2001
2 Введите значение x (или 'q' для выхода): 2222
3 Время выполнения: 0.0008919239044189453 секунд

```

Пример ввода некорректного значения:

```

1 Введите количество итераций (или 'q' для выхода): 2001
2 Введите значение x (или 'q' для выхода): g
3 Ошибка: введено не число. Завершение программы.

```

Таким образом, реализация на Python выполняет ту же задачу, что и версия на C++, однако с использованием интерпретируемого языка, что сказывается на производительности. При сравнении с C++-версией наблюдается примерно в 4 раза большее время исполнения на аналогичном объёме вычислений.

Сравнительный анализ производительности

Результаты замеров показывают, что время выполнения вычислений на Python существенно превышает аналогичный показатель на C++. Причины этого заключаются в фундаментальных различиях между языками программирования и механизмами их выполнения:

- **C++ — компилируемый язык:** код транслируется в машинные инструкции до выполнения, что позволяет достичь высокой скорости исполнения за счёт прямого обращения к ресурсам процессора.
- **Python — интерпретируемый язык:** каждая строка кода анализируется и выполняется в момент запуска, что создаёт дополнительную нагрузку на процессор и увеличивает общее время выполнения.

- **Оптимизация:** компилятор C++ способен применять низкоуровневые оптимизации, такие как распараллеливание, удаление мёртвого кода, векторизация и др. В Python эти оптимизации недоступны без сторонних средств (например, Numba, Cython).
- **Garbage Collector:** в Python встроена автоматическая сборка мусора, что вносит дополнительные издержки на управление памятью.

Таким образом, для ресурсоёмких задач, критичных к производительности, C++ обеспечивает значительно лучшее быстродействие.

Структура и фиксация проекта в Git

В рамках данной лабораторной работы:

- Создана структура проекта с каталогами `src`, `build`, `doc`;
- В папке `doc` сохранён отчёт по лабораторной работе (в формате PDF и `.tex`);
- Все изменения зафиксированы через ветку `dev`;
- С помощью `bash`-скриптов ревизии перенесены на ветки `stg` и `prd`;
- Созданы вспомогательные скрипты для отката состояния репозитория к последнему коммиту.

Создание скриптов

Переходим в корневую директорию проекта и создаём скрипты:

```
1 cd cpp_unix_project/lab_02
2 touch deploy_to_stg.sh deploy_to_prd.sh reset_to_last_commit.sh
3 chmod +x deploy_to_stg.sh deploy_to_prd.sh reset_to_last_commit.sh
```

Содержимое скриптов

`deplotostg.sh` — накатить `dev` на `stg`:

```
1 #!/bin/bash
2 git checkout stg
3 git merge dev --no-ff -m "Deploy dev to stg"
4 echo "Ревизия успешно перенесена с dev на stg"
```

`deplotoprd.sh` — накатить `stg` на `prd`:

```
1 #!/bin/bash
2 git checkout prd
3 git merge stg --no-ff -m "Deploy stg to prd"
4 echo "Ревизия успешно перенесена с stg на prd"
```

`resettolastcommit.sh` — откат текущих изменений:

```
1 #!/bin/bash
2 git reset --hard
3 git clean -fd
4 echo "Откат к последнему коммиту выполнен. Несохранившиеся изменения удалены."
```

Все скрипты размещены в корне папки `lab02`, (`chmod +x`).

Пример выполнения скриптов

```
1 $ ./deploy_to_stg.sh
2 Переключено на ветку 'stg'
3 Merge made by the 'recursive' strategy.
4 Ревизия успешно перенесена с dev на stg
5
6 $ ./deploy_to_prd.sh
7 Переключено на ветку 'prd'
8 Merge made by the 'recursive' strategy.
9 Ревизия успешно перенесена с stg на prd
10
11 $ ./reset_to_last_commit.sh
12 Откат к последнему коммиту выполнен. Несохранившиеся изменения удалены.
```